# WHACK-A-MOLE GAME DEVELOPMENT USING PYTHON

## AIM

To develop a Whack-a-Mole game using Python's Tkinter library in which the player clicks the randomly appearing mole to score points within a limited time.

## PROCEDURE

1. Import the Tkinter and random modules.
2. Set the game time limit and mole display interval.
3. Create the main game window and configure its appearance.
4. Display the title, score, timer, and message labels.
5. Create nine buttons arranged in a 3 × 3 grid to represent the holes.
6. Randomly display the mole in one of the holes.
7. Increase the score when the player clicks the correct hole.
8. Update the timer every second and stop the game when the time expires.
9. Provide restart and exit buttons.
10. Run the application using Python.

## PROGRAM CODE

```
import tkinter as tk
import random

# Game Settings
TIME_LIMIT = 30
MOLE_DELAY = 800


class WhackAMole:
    def __init__(self, root):
        self.root = root
        self.root.title("WHACK-A-MOLE")
        self.root.resizable(False, False)
        self.root.configure(bg="#172554")

        tk.Label(
            root,
            text="WHACK-A-MOLE",
            font=("Arial Black", 22),
            bg="#172554",
            fg="white"
        ).pack(pady=10)

        self.score_label = tk.Label(
            root,
            text="SCORE: 0",
            font=("Arial", 16, "bold"),
            bg="#172554",
            fg="#FACC15"
        )
        self.score_label.pack(pady=5)

        self.timer_label = tk.Label(
            root,
            text="TIME: 30",
            font=("Arial", 14, "bold"),
            bg="#172554",
            fg="white"
        )
        self.timer_label.pack(pady=4)

        self.message_label = tk.Label(
            root,
            text="Click the mole!",
            font=("Arial", 12),
            bg="#172554",
            fg="#BAE6FD"
```

```
)
self.message_label.pack(pady=4)

self.game_frame = tk.Frame(
    root, bg="#172554"
)
self.game_frame.pack(padx=15, pady=10)

self.holes = []

for i in range(9):
    button = tk.Button(
        self.game_frame,
        text="HOLE",
        width=10,
        height=3,
        font=("Arial", 10, "bold"),
        bg="#854D0E",
        fg="white",
        command=lambda n=i: self.hit_mole(n)
    )
    button.grid(
        row=i // 3,
        column=i % 3,
        padx=6,
        pady=6
    )
    self.holes.append(button)

button_frame = tk.Frame(
    root, bg="#172554"
)
button_frame.pack(pady=10)

tk.Button(
    button_frame,
    text="RESTART GAME",
    font=("Arial", 10, "bold"),
    bg="#22C55E",
    fg="black",
    command=self.restart
).pack(side="left", padx=8)

tk.Button(
    button_frame,
    text="EXIT",
    font=("Arial", 10, "bold"),
```

```
            bg="#EF4444",
            fg="white",
            command=root.destroy
        ).pack(side="left", padx=8)

        self.mole_position = None
        self.score = 0
        self.time_left = TIME_LIMIT
        self.running = False
        self.mole_timer = None
        self.clock_timer = None

        self.restart()

    def restart(self):
        if self.mole_timer is not None:
            self.root.after_cancel(self.mole_timer)

        if self.clock_timer is not None:
            self.root.after_cancel(self.clock_timer)

        self.score = 0
        self.time_left = TIME_LIMIT
        self.mole_position = None
        self.running = True

        self.score_label.config(text="SCORE: 0")
        self.timer_label.config(text="TIME: 30")
        self.message_label.config(text="Click the mole!")

        for button in self.holes:
            button.config(
                text="HOLE",
                bg="#854D0E",
                fg="white",
                state="normal"
            )

        self.show_mole()
        self.update_timer()

    def show_mole(self):
        if not self.running:
            return

        new_position = random.randrange(len(self.holes))
```

```
        while new_position == self.mole_position:
            new_position = random.randrange(len(self.holes))

        self.mole_position = new_position

        for i, button in enumerate(self.holes):
            if i == self.mole_position:
                button.config(
                    text="MOLE!",
                    bg="#F97316",
                    fg="black"
                )
            else:
                button.config(
                    text="HOLE",
                    bg="#854D0E",
                    fg="white"
                )

        self.mole_timer = self.root.after(
            MOLE_DELAY, self.show_mole
        )

    def hit_mole(self, position):
        if self.running and position == self.mole_position:
            self.score += 1

            self.score_label.config(
                text="SCORE: " + str(self.score)
            )
            self.message_label.config(
                text="Good hit!"
            )

            self.mole_position = None

            for button in self.holes:
                button.config(
                    text="HOLE",
                    bg="#854D0E",
                    fg="white"
                )

    def update_timer(self):
        if not self.running:
            return
```

```
        if self.time_left > 0:
            self.time_left -= 1

            self.timer_label.config(
                text="TIME: " + str(self.time_left)
            )

            self.clock_timer = self.root.after(
                1000, self.update_timer
            )
        else:
            self.game_over()

    def game_over(self):
        self.running = False

        if self.mole_timer is not None:
            self.root.after_cancel(self.mole_timer)

        if self.clock_timer is not None:
            self.root.after_cancel(self.clock_timer)

        self.mole_timer = None
        self.clock_timer = None
        self.mole_position = None

        for button in self.holes:
            button.config(
                text="HOLE",
                bg="#854D0E",
                state="disabled"
            )

        self.message_label.config(
            text="GAME OVER! Final score: "
            + str(self.score)
        )


# Main Application
root = tk.Tk()
game = WhackAMole(root)
root.mainloop()
```

output:







RESULT:

Thus, the Whack-a-Mole game was developed successfully using Python Tkinter. The game displays a randomly appearing mole, tracks the player's score, maintains a countdown timer, and provides restart and exit options.